Software Construction and Decision Making

Alejandro Benítez

Class Notes

February 09, 2026

who knows what will be here

Contents

1. Socioformadores 6
   1.1. Mareé 6
   1.2. Necesidad 6
   1.3. Módulos 6
      1.3.1. Menú 6
      1.3.2. Administración 6
      1.3.3. Experiencia 6
      1.3.4. Diseño 7
      1.3.5. Lealtad 7
      1.3.6. Métricas 7
   1.4. Preguntas 7
   1.5. Notas 7
2. Project Management 8
   2.1. An Introduction 8
      2.1.1. Projects and Project Management 8
      2.1.2. PMBOK 8
         2.1.2.1. Project Integration Management 9
         2.1.2.2. Project Communication Management 9
         2.1.2.3. Project Scope Management 9
         2.1.2.4. Project Schedule and Cost Management 9
         2.1.2.5. Project Quality and Risk Management 9
         2.1.2.6. Project Resource and Procurement Management 9
   2.2. A Review on Objectives 9
   2.3. Scope 10
      2.3.1. Initiation 10
      2.3.2. Scope Planning 11
   2.4. Scope Definition 11
   2.5. Work Breakdown Structures 11
   2.6. Context Diagram 11
   2.7. Project Charter 11
   2.8. Comunicación 12
      2.8.1. Identificar a los interesados 12
      2.8.2. Planear la comunicación 13
      2.8.3. Distribuir la información 13
      2.8.4. Administración de las expectativas 13
      2.8.5. Reportar el diseño 14
   2.9. Addendum 14
3. Information Systems 15
   3.1. An Introduction 15
      3.1.1. Definitions 15
      3.1.2. Architecture 15
   3.2. DB & DBMS 16
      3.2.1. DBMS 16

3.2.2. Objetivos ... 17
3.3. Notación Modelo Entidad Relación (MER) ... 17
3.3.1. Entidades ... 17
3.3.2. Asociaciones ... 17
3.3.3. Atributo ... 18
3.3.4. Metodología ... 18
3.4. MER Extendido ... 18
3.4.1. Roles ... 18
3.4.2. Relaciones ISA ... 19
3.4.3. Entidades fuertes y débiles ... 19
3.4.4. Restricciones de integridad adicionales ... 19
3.5. Traslado de MER a Tablas ... 19
3.5.1. Procedimiento de Transferencia ... 19
3.5.2. Procedimineto para MER Extendido ... 20
3.6. Procedimiento para Elaborar una BD ... 20
3.7. Álgebra Relacional ... 20
3.7.1. Llaves ... 21
3.7.2. Operaciones ... 21
3.7.3. Alg Relacional ... 21
3.8. SQL ... 22
3.9. Stored Procedures and Functions ... 23
3.9.1. Funciones vs Procedimientos ... 23
3.9.2. Ventajas y Desventajas ... 24
3.10. Transacciones ... 24
3.10.1. Atomicidad ... 25
3.10.2. Coherencia ... 25
3.10.3. Aislamiento ... 25
3.10.4. Durabilidad ... 25
3.11. Normalizacion ... 25
3.11.1. 1FN ... 25
3.11.2. 2FN ... 26
3.11.3. 3FN ... 26
3.11.4. Demas Formas ... 26
4. UML ... 27
4.1. Diagramas de Secuencia ... 27
4.1.1. Objetos ... 27
4.1.2. Flujos ... 28
4.1.3. Operadores ... 29
4.2. Generar Pruebas a Partir de Casos de Uso ... 30
4.2.1. General los Escenarios ... 30
4.2.2. Identificar los Casos de Prueba ... 31
4.2.3. Identificar Valores ... 31
4.3. Diagrama de Estado ... 32
4.4. Diagramas de Despliegue ... 32
5. Entrevistas ... 34

Index of Figures 35

1. Socioformadores

1.1. Mareé

Mauricio Hernández Siles (442 443 5874), Joel, María. @mareecrepe

Crepería. Experiencia completa.

Proyecto en forlatecer la relacion con su comunidad. Incrementar la lealtad que tienen.

- Sistema de royalty para clientes. Mobile. Tarjeta digital.
- Menu digital, personalizado. Horarios y disponibilidad. Futuro, pedir comida en la aplicación.
- Portal administrativo.
- Notificaciones whatsapp, programar mensajes.
- Adaptible y disponible en los telefonos.

Quieren una solución integral personal. Evitar limitantes.

Apoyo con el diseño.

1.2. Necesidad

El objetivo principal es fortalecer la relación con su comunidad, e incrementar la lealtad de los consumidores. La experiencia del cliente es muy importante.

1.3. Módulos

1.3.1. Menú

Debe de ser muy customizable, dependiendo de lo que quiero y que le quiero poner el menu cambia. Yo puedo armar mi propia crepa. Incluso manejar toppings o ingredientes de temporada. Debe de haber dos modelos de menu de crepas

- Arma tu crepa
- Crepas predefinidas que pueden ser modificadas

Proveen cafe, crepas, y waffles.

1.3.2. Administración

Poder modificar precios, ingredientes, usuarios, de temporada, horarios de atención,

1.3.3. Experiencia

Primero era un modelo *to go*, pero no funcionaba tan bien. Ahora funciona como un restaurante, llegan, menus, comandean.

Si le gustaría poder educar a su cliente a poder ordenar por medio de una aplicación.

Son mayormente mujeres, y si son hombres son es porque vienen con su pareja. Es en un casona histórica de Mazatlán.

No quiere cambiar su tipo de venta. Hay un prepa cerca y muchos van, por lo que estaría bien que los chavos puedan utilizarlo. No se quiere alejar del cliente, se quiere acercar por medio de la aplicación.

Saber del cliente, saber su nombre, si se fue de viaje, tener ese acercamiento con el cliente y la aplicación debe poder complementar eso.

Pedidos caen a un sistema que hace un sonidito especial, y manualmente se ingresa a la comanda. Tener una ipad a full volumen para reconocer que es del sistema. Sería bien tener un pedido que sepa quien estaba recibiendo esas comandas para saber si hubo algún error.

1.3.4. Diseño

Girly, enfocado a la mujer. Pero, fácil de usar y amigable a los hombres.

1.3.5. Lealtad

Tienen una tarjetita que va puntuando. Llevar el control de saber que consume, invitarlo a volverlo a consumir. Se tiene que tener información para poder dar recomendaciones y mensajes estratégicos. Tiene que ser personalizado, por ejemplo descuentos en cumpleaños. En consumo de x cantidad puedes participar en rifas, etc.

1.3.6. Métricas

Poque no se mueven algunas crepas. De dinero no se necesita mucho. Se enfoca más en la visita del cliente, si se la pasó bien, etc.

1.4. Preguntas

- No me queda muy claro a que se refieren con un portal administrativo.
- Que tipo de recompesa quieren ofrecer, puntos, descuentos, productos gratis
- La idea es que hayan membresias
- El menu cambia segun horario

1.5. Notas

Va a haber muchas oportunides para conectar con terceros. Apple pay, google wallet, whatsapp.

2. Project Management

2.1. An Introduction

The course seeks to teach techniques and knowledge for the efficient project management. It is mainly based on the Project Management Institute (PMI) and its research.

First it is important to realize that few are the projects that are successful. Only about 17% of all projects can be considered successful. Even optimistic estimates only reach about 20 to 30%.

There are some points of suspicion, points which can give us insight into why projects fail so often. These are but some of them:

- Incomplete or changing specifications and requirements
- Lack of user involvement
- Not enough technical knowledge
- Inadequate use of methods and tools
- Unrealistic expectations

We can conclude the main reason is **weak project management**.

2.1.1. Projects and Project Management

But first let us define what a project even is. A project is a temporal effort in order to bring a product, service or result to life. This definition implicates three things: effort, which is the use of organized resources; time constrainss, for it must be completed at some point; and the product itself, which must be alive, functioning.

Knowing this we might be able to answer, what is project management then?

> Project management is the exercise of knowledge, abilities, tools and techniques in the activities that comprise a project in order to meet the project requirements.

2.1.2. PMBOK

Now PMI, a non lucrative association, is the author and proprietary of the **PMBOK**, the most complete manual on project management. It establishes standards, it homogenizes training programs and even certificates individuals. And PMBOK is its Project Management Body of Knowledge, which, among other things, describes the environment wherein the projects will be developed, describes and organizes the project characteristics, describes the necessary knowledge to manage a project (divided into 9 areas), and establishes a shared vocabulary.

The PMBOK manages mainly 9 areas, namely:

- Project Integration Management
- Project Communication Management
- Project Scope Management
- Project Schedule Management

- Project Cost Management
- Project Quality Management
- Project Risk Management
- Project Resource Management
- Project Procurement Management

2.1.2.1. Project Integration Management

Includes the necessary processes to guarantee that the distinct elements of the project are well coordinated. It includes the development ofo the project plan, the execution of said plan, a control for the changes made. This is a recursive process. A plan is made, then executed and revised by the control. With the feedback from the control the plan is altered, then executed and so on. Once the control gives a green light the process is terminated.

2.1.2.2. Project Communication Management

It deals with how to generate, collect, distribute, store, recover and delete the projects information.

2.1.2.3. Project Scope Management

How to determine if the projects contains and only contains the necessary work to execute the project. How to control de use of resources and execution of activities so that the project may have the exact desired characteristics and functionalities.

2.1.2.4. Project Schedule and Cost Management

How to determine the order and sequence of activities to deliver in time. How to assign resources so that the project stays within budget. How to use the resources, time and money.

2.1.2.5. Project Quality and Risk Management

Quality deals with how to satisfy the project necessities by the continued improvement of processes. Risk deals with how to identify events that are out of the project control, and how to act accordingly.

2.1.2.6. Project Resource and Procurement Management

Resource means how to organanize and adminitrate the team for the project, balancingn participaation, desicion taking and commitment. Procurement means how to adquire the goods and services required by the project, as well as knowing how to select from providers.

2.2. A Review on Objectives

Any project must divided in objectives. But first it is important to identify the necessity or problem, the **pain points** of the client. Only afterwards can the objectives be identified and detailed.

- Objective (usually in text)
- Functional (usecases diagrams)
- Non functional (quality attributes)
- Information (catalogues, fields, relations)
- Business rules

- Interface
  - User GUI
  - Communication (between other systems)

Extensible is a system that is able to grow by implementing more. Scalable is that the system grows just by inputing more computer power.

Question: How relevant is it nowadays the PMBOK and are big companies actually using it?

2.3. Scope

It is composed of all, and only those, necessary activities, works and processes that guarantee a successful project. It includes:

- Initiation
- Scope planning
- Scope definition
- Scope verification
- Scope change control

The scope is the aggregate of the *products* and *services* that need to be delivered, and defines what should be included but also what should not be included. These objectives are written in a SMART format.

But before diving into this subject, let's differentiate between *product scope* and *project scope*.

Product scope deals with the characteristics and functionalities that should be included in a given product of system. The success of it is verified with the requirements (functional, nonfunctional, business, etc.).

On the other hand, project scope deals with the work and tasks that need to be done in order to deliver the product with said characteristics and functionalities. Its success is verified with th project plan.

2.3.1. Initiation

Is the formal authorization of a new project of phase. It guarantees organizational support for the project. Among other things it defines the high level objectives, secures the necessary resources and approvals, confirms the alignment of the project with the strategic objectives, and assigns a project boss.

Since there may be many project ideas, only the best one should be considered. Therefore a method of priorization is necessary. The available methods include: focusing on general necessities, cathegorizing the projects, financial analysis, and punctuation methods.

Once the project has been selected it must be formalized so that everyone involved in the project is on the same page. This formalization must contain:

- Objectives
- Success criteria
- Restrictions
- Assumptions

- Roles and responsibilities

Is this formalization the same as the project charter or a separate document?

There must be a project charter that recognizes the project and informs about the necessities of the project and the description of the product. This charter must have authority, and therefore must be signed by a high ranking official (xd).

2.3.2. Scope Planning

The development of a written scope statement that will serve as the foundation for future decisions regarding the project. It also establishes an agreement between the project and client in order to identify both the objectives as well as the things that will be delivered.

This statement must include (a) a project justification, (b) the project product, (c) the things to be delivered, and (d) the project objectives (written in a SMART method).

The scope management plan must describe how de project will be managed and how scope changes may be integrated. It is part of the global project plan. It must include the estability expectations and a clear description as to how scope changes will be identified and classified.

2.4. Scope Definition

Now the work must be divided into several more manageable pieces. Here we seek to

- Improve the presicion of estimates for time, cost, and resources.
- Define the baseline by which the project will be compared and controlled.
- Help clearly define the responsibilities of each task.

A deliverable is every produc, result or measurable element, tangible and verificable that will be delivered.

2.5. Work Breakdown Structures

no idea what this is

It is the process of dividing the deliverables and work in smaller and more manageable products. It is a hierarchical decomposition delivery oriented.

A work package is the lowest level. We now our division has gotten there when we can (a) state when it begins and ends, (b) estimate costs and efforts, and (c) monitor and control it.

The detail level must be enough.

2.6. Context Diagram

It helps us identifying the scope of the project, the people iterested and how data flows through the system.

2.7. Project Charter

Must include a description of the project, the people responsible, budget and time, what should be in the project and what not.

There must also be milestones that help divide the project and get tangible results, as well as knowing exactly how bad you are developing.

2.8. Comunicación

La comunicación es otro aspecto realmente importante a la hora de gestionar proyectos. Hay varias metodologías para realizar esta tarea, pero se pueden identificar los siguientes pasos.

- Identificar a los interesados
- Planear la comunicación
- Distribuir la información
- Administrar las expectativas de los interesados
- Reportar el desempeño

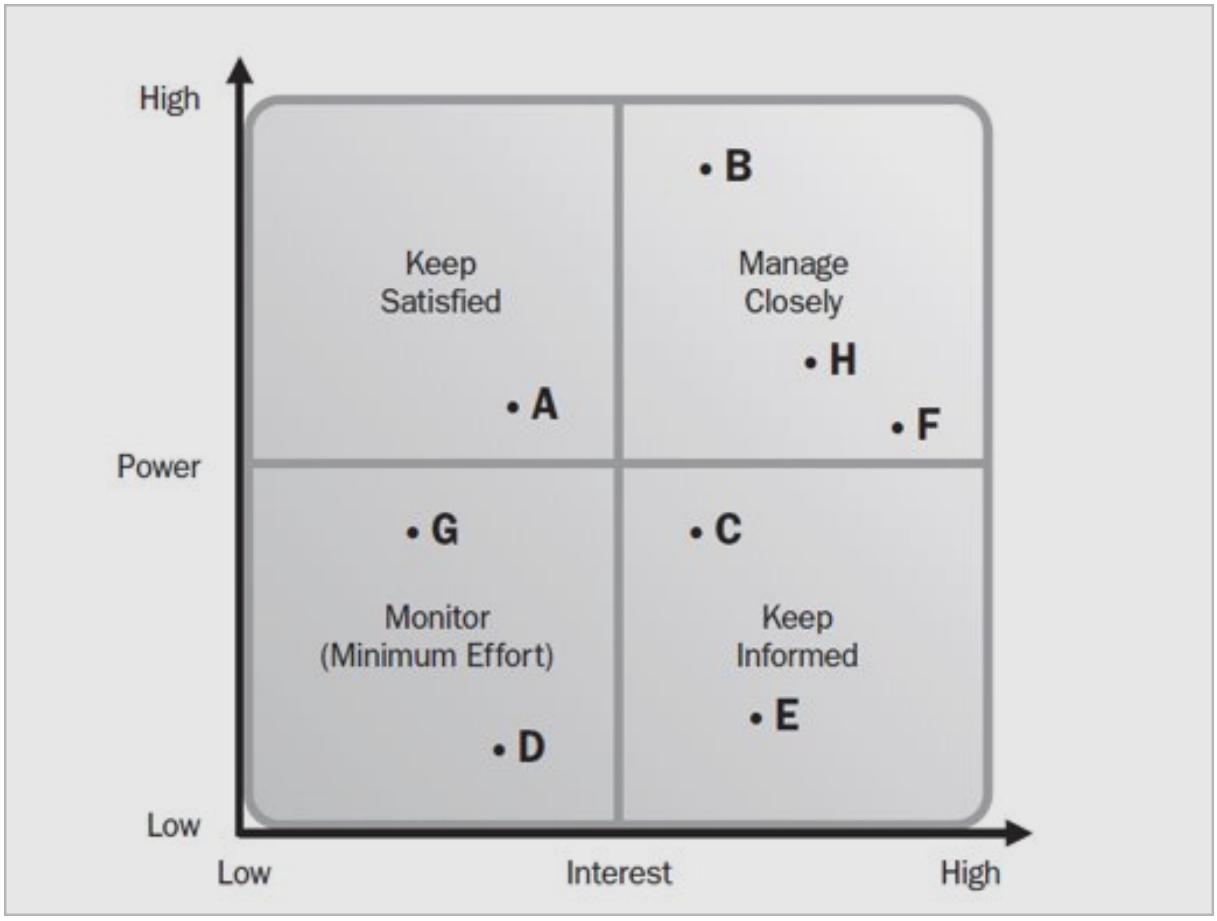
La comunicación tiene varias dimensiones. Existe la comunicación interna entre los colaboradores del proyecto y la externa entre el equipo y los interesados. Existe la comunicación formal y la informal, así como la oficial y la no oficial. También está la verbal y la no verbal, y la escrita y oral.

Algunos ejemplos de esto pueden ser la buena escucha, la investigación, la validación de datos, la **administración de expectativas**, la persuación y negociación, resumir y parafrasear lo que entendemos, y también proponer ideas y los siguientes pasos.

2.8.1. Identificar a los interesados

Este es un paso crítico y debe de ser realizado lo antes posible en el desarrollo del proyecto. Esto nos permite conocer realmente que es lo que quieren y conocer sus intereses para poder desarrollar una estrategia para atender sus necesidades. La idea es **maximizar la influencia positiva** y **minimizar la resistencia**.

Para esto es muy útil utilizar una matríz de los interesados donde se especifique que tanto poder o autoridad tienen en el proyecto, así como cuanto interés tienen en este.



Sobre todo es importante definir estrategias para poder ganar su apoyo o reducir su resistencia.

2.8.2. Planear la comunicación

Este paso también es crucial. Es muy complicado que se puedan conseguir los objetivos de comunicación si no se tiene un plan de comunicación, ya que no todos cuentan con el mismo compromiso ni los mismos tiempos.

Este plan busca responder a preguntas como ¿quién necesita la información?, ¿cuándo necesita la información?, ¿cómo se le van a entregar? y finalmente ¿quién la va a entregar?

2.8.3. Distribuir la información

Ahora, también es muy importante saber la forma en la que se va a entregar esa información. Esto es la forma de las reuniones, las presentaciones, pero también engloba el estilo que se va a utilizar. La idea es que exista una estandarización para que no sea complicado recordarlo y además sea consisente para no requerir tener que adaptarse a varios formatos distintos.

También se debe de almacenar la retroalimentación de los interesados, así como las lecciones aprendidas.

2.8.4. Administración de las expectativas

Esto es para atender las necesidades de los interesados. También incluye la comunicación para negociar y manejar las expectativas de los interesados, la explicación y aclaración de las inquietudes de una forma temprana, y también la resolución de problemas como son las propuestas de cambio.

2.8.5. Reportar el diseño

Finalmente es importante reportar el desempeño. Esto se logra con reportes de estado, mediciones de avance y estimaciones. También es importante tener una comparación de estas estimaciones contra el avance actual.

También es importante dar a conocer los pasos que siguen, es decir lo que el equipo tiene planeado hacer después (como una sección de next steps).

Pregunta: Que literatura existe para aprender esto aparte del pmbok?

2.9. Addendum

- The people **interested** in the project are those who benefit and are affected by it.
- **Project agreement** is the document that formalises what the goals, scope and overall things that will be done in the project, which includes the time. It must also outline what the project will contain and what it will not.

3. Information Systems

3.1. An Introduction

> An information system is defined as a set of interrelated components that compile, process, store and distribute information with the goal of decision taking and control over organization.

3.1.1. Definitions

- **Data:** Is the symbolic representation of an attribute or characteristic of an entity. It does not have semantic meaning but may be conveniently processed into a more valueable thing.
- **Information:** Is an organized set of data that has been processed and thus has semantic meaning.
- **Input:** Is the information (data?) produced by the user. An input also can be produced by networks, etc.
- **Processing:** Are the actions that take an entry and generate useful information for the user.
- **Output:** Is the information that the computer will show to the user.
- **Feedback:** Is the added value that allows users to take actions.

3.1.2. Architecture

It is divided in mainly five levels (types), each one with a specific user in mind.

1. **Transaction Processing System (TPS):** Help in the daily activities of a enterprise. They are the heart of the information system. It comprises the data collection, data manipulation, storage, and report production.

   Its goal is to process data, be exact and correct, mantain data integrity, produce oportune reports and documents, and overall improve laboral efficiency. It, thus, has a big volume I/O, needs high storage capabilities, and impacts a high number of user (specially when there is an error and everyone is stroke).

   A transaction means sort of CRUD, create, recover, update and delete information. There are three processing types:

   - In line: In the moment as it arrives, though it does not matter if it lags behind (register user).
   - In batch: Waits a bit to gather up and then processes.
   - Realtime: Processes instantly as it arrives, where the time is critical (air traffic).

2. **Office Automation Systems (OAS):** Provide help for daily activities to increase employee efficiency of those who work with the information (e.g. word processors, email, schedules, etc.).

Knowledge Work Systems (KWS): Provide help for specialized employees in the creation and integration of new knowledge for the institution, for instance software that aids architects in blueprint design.

3. **Management Information Systems (MIS):** A collection of people, procedures, databases, and devices needed to display information to the managers and decision takers so that they may achieve the objectives. They seek to show a general vision of how things look and how and which operations take place.

   These systems produce reports, which may be scheduled, demanded or exceptional (when a critical situation creates the need).

   Decision Support Systems: It is similar to a MIS, but its purpose is to take decisions regarding specific problems, semi-structured problems or non-structured ones. It also gathers information from many places, but it must also be able to perform complex and sophisticaded analysis using statistical packages. It must also be visual. A DSS serves as a *what if* visualizer, allowing the user to see the impact of hypothetical cases.

4. **Group Decision Support Systems (GDSS):** This system aids group decision taking. Its design is special, as it must encourage creative thinking and effective communication. It must also be easy to learn and use, while also being flexible enough to acommodate for the different kinds of people. It should also have some degree of anonymity in order for to avoid source bias.

   Expert Systems (ES): A system that acts as a human expert in certain field or area. It helps identifying issues, predicting future events and aiding in the design of new products.

5. **Executive Information Systems (EIS):** Encompasses all critical information that influences decision taking. It aids non structured decision taking with KPIs graphicly. It uses quality information, reduced use of the keyboard, secure access, fast response time, remote access and perfect fitted disign.

3.2. DB & DBMS

Se puede pensar en una base de datos como una colección de archivos interrelacionados logicamente, cuyos datos han sido cuidadosamente diseñados para que estos archivos contengan la menor repetición de datos y hechos.

El uso de una base de datos es muy conveniente ya que nos permite trabajar con una mayor cantidad de datos, además de que se le puede dar una estructura bien diseñada para facilitar su uso.

3.2.1. DBMS

Un sistema de gestión de base de datos consiste en una colección de datos y un conjunto de programas para controlar esos datos. A la colección de datos interrelacionados se le denomina como la base de datos, y al conjunto de programas como el DBMS, que crea un entorno conveniente y eficiente para extraer y almacenar información de la base de datos.

3.2.2. Objetivos

- Eliminar la redundancia y la inconsistencia de los datos. El hecho de tener redundancia solo aumenta los costos de almacenamiento y acceso. Además, puede llevar a una inconsistencia de datos donde los diferentes registros contienen diferente información y no concuerden entre sí.
- Elimina la dificultad de acceso a la información. Las bases de datos proporcionan un sistema conveniente y eficiente para recuperar información, pero que a la vez es lo suficientemente generales para poder ser utilizado en varios lugares.
- Los datos están naturalmente relacionados.
- Seguridad al acceder de manera concurrente.
- Seguridad para evitar que cualquiera pueda acceder a la información.
- Integridad de los datos. Se pueden añadir restricciones a los datos dentro de la base de datos para asegurar que ciertas reglas se cumplan.
- Crear copias y backups de la base de datos.:

3.3. Notación Modelo Entidad Relación (MER)

Es un método y diagrama que nos permite diseñar bases de datos de una buena manera. Un modelo entidad relación se compone de ciertos elemetos, los cuales son explicados a continuación.

Es un modele de análisis de apoyo a la elaboración de modelo de persistencia. En este modelo siempre estoy pensando en el tiempo.

3.3.1. Entidades

Representan el objeto básico de nuestro diagrama, y se refieren a cualquier objeto del mundo real sobre el cual queremos tener información. Se representa como un rectángulo.

3.3.2. Asociaciones

Representan la interrelación que existe entre las entidades del modelo.

Se debe de procurar que los nombres de las asociaciones sean precisos, usualmente un solo verbo que describe la relación entre las entidades involucradas. Una asociación representa solo una forma en la que las entidades se relacionan, pero puede que dos entidades se relacionen de varias formas, por lo que de debe de especificar las distintad asociaciones que hay entre esas entidades.

Adicionalmente, una asociación puede tener atributos descriptivos, lo que nos permite almacenar información extra acerca de el tipo de relación que se tiene.

El grado de una asociación se refiere a la cantidad de entidades que participan en la relación. Esto también quire decir que se pueden tener varias entidades participando en una sola asociación. Sin embargo es aconsejable restringirlo a solo relaciones binarias.

La cardinalidad es el número de elementos que participan en una asociación. Existen tres casos generales: asociaciones uno a uno (1:1), uno a muchos (1:N), y relaciones muchos a muchos (N:M). Para casos más específicos se puede establecer explícitamente el valor de N.

Existen dos tipos de participación en una asociación. Primero se encuentra la parcial u opcional. Dentro de esta categoria no todas las instancias de una entidad tienen que estar relacinadas con la otra entidad. Se representan con una sola línea. Por otra parte, las asociaciones oblgatorias o parciales deben de estar siempre relacionadas. Se dice que existe una participación total. Se representan con dos líneas.

3.3.3. Atributo

Los atrbutos son la información que nos interesa recolectar. Son características y valores. Se le pueden representar como elipses y como una lista de los nombres de los atributos.

El conjunto de valores que puede tomar un atributo se le llama dominio.

Existen varios tipos de atributos:

- **Simples o compuestos:** Los compuestos están formados por varios atributos, los almacenados no.
- **Mono valuados o multivariados:** Los mono solo pueden almacenar un valor, los multi varios valores.
- **Almacenados y derivados:** Los almacenados son almacenados. Los derivados no son almacenador, sino que se pueden obtener mediante cálculos y funciones sobre otros atributos que sí son almacenados.

Un atributo muy importante y especial es el identificador de identidad. Puede ser uno o más atributos que identitican a una identidad como única.

3.3.4. Metodología

1. Identificar entidades
2. Incorporar atributos
3. Determinar identificadores
4. Identificar asociaciones
5. Determinar cardinalidad
6. Incorporar atributos en las asociaciones
7. Verificar con los requerimientos

3.4. MER Extendido

Estos son elemetos que buscan expandir y complementar el modelo relación entidad. Sin embargo se debe de tomar en cuenta que estos elementos no se deben de abusar, deberían de considerarse como excepciones y no siempre deben de estar presentes, muchas veces el modelo no extendido es suficiente.

3.4.1. Roles

Se utilizan cuando una entidad juega más de un rol en una asociación, ya sea consigo misma (recursiva) o con múltiples entidades. Se representan como un rectángulo pegado a la entidad.

3.4.2. Relaciones ISA

Utilizan entidades generalizadoras (superclase), y entidades especializadoras (subclases).

- Una **superclase** es todo tipo de entidad a la que se le definen subclases.
- Una **subclase** es una entidad que cuenta con todos los atributos de la superclase, pero adicionalmente cuenta con atributos extra.

Estas son entidades distintas, por lo que un objeto no puede jugar el rol de ambas, a diferencia de un rol donde esto sí sucede. La forma en la que estas entidades operan es muy similar a la herencia en el paradigma de programación irientada a objetos.

3.4.3. Entidades fuertes y débiles

- **Fuertes:** Son las entidades normales. Existen por sí mismas.
- **Débiles:** Son las entidades que dependen de otras. Si la entidad en la que dependen desaparece, entonces la intidad débil también lo hace. Se identifican con un doble rectángulo.

3.4.4. Restricciones de integridad adicionales

- **Cotas de cardinalidad:** Son una extensión de la notación de cardinalidad. Se añade información acerca del número mínimo y máximo que pueden haber de asociaciones. Se delimitan de la forma `min:max`.
- **Entidad atributo:** Son restricciones adicionales que existen entre lor atributos de una endidad. Algunos ejemplos de estas son restricciones de recursos humanos, ecológicas, de tiempo, instalaciones, transporte, recursos financieros, materiales, etc.

¿Cómo se especifican estos dentro del MER?

3.5. Traslado de MER a Tablas

Para poder llevar nuestro modelo a la realidad es necesario hacer una traducción del modelo entidad relación a algo más concreto, que en este caso llamaremos modelo relacional. Tal modelo se definirá de la siguiente manera:

```
nombre(columna1, columna2, columna3, ...)
```

Para reconocer la llave primaria de la tabla esta se subrayará, mostrando que la columna o conjunto de columnas representan una llave.

3.5.1. Procedimiento de Transferencia

Para cada entidad se define una **tabla** cuyas **columnas** corresponden uno a uno a los atributos de la entidad. De no existir un identificador en la tabla que garantice unicidad entonces se debe de agregar.

Para las asociaciones con **cardonalidad 1:1** solo se tiene que agregar la llave primaria de una tabla en la otra. Es indistinto cual llave es agregada.

Para las asociaciones con **cardinalidad 1:N**, se debe de agregar la llave primaria de la entidad que ocurre varias veces en la entidad que solo ocurre una vez. Por ejemplo, si un profesor puede dar varios grupos pero solo uno puede dar un grupo, entonces la llave primaria del profesor debería de estar en cada grupo.

Finalmente, para **cardinalidades de tipo N:N** se debe de crear una nueva tabla donde se encuentran las dos llaves primarias de ambas entidades, además de los atributos extra que se requieran. La llave primaria de esta tabla sería la concatenación de las llaves primarias de ambas entidades.

3.5.2. Procedimineto para MER Extendido

Las entidades con **relaciones ISA** son 1:1, sin embargo su mapeo es un caso particular de las relaciones 1:N. La llave primaria se encuentra en la entidad generalizadora (superclase o clase padre), y las entidades especializadoras (subclase o clase hija) simplemente comparten exáctamente la misma llave primaria. Se dice que las entidades generalizadoras heredan su llave primaria a las entidades especializadoras.

Para el caso de las **entidades débiles**, la entidad fuerte de la que dependen le hereda su llave primaria, similar al caso anterior. Sin embargo, tal entidad débil ya tiene un identificador único, por lo que su llave primaria se vuelve la concatenación de la llave primaria heredada con el identificador original.

Finalmente, el caso de los **roles** es muy similar. Secillamente se considera que es la misma entidad independiente del rol, y dentro de la llave foránea (que sería la llave primaria de una intidad igual) se especifíca el rol que tal entidad foránea juega. Para asociaciones de N:N se sigue el mismo procedimiento, en donde se crea una tabla extra.

3.6. Procedimiento para Elaborar una BD

1. Elaborar diagrama de contexto
2. Elaborar MER (análisis, modelo conceptual)
3. Traducir a ER (diseño, modelo lógico)
4. Construir la BD (construcción, modelo físico)
5. Cargar tablas (a mano, importar archivos, automatizar registros)
6. Convertir datos en información (generar reportes)

3.7. Álgebra Relacional

El álgebra relacional es una rama de las matemáticas aplicada en las bases de datos relacionales, ya que nos permite manejar conjuntos de datos.

Existen algunos conceptos importantes:

$$\begin{aligned} A &= \{a, b, c\} \\ B &= \{\heartsuit, \spadesuit, \clubsuit, \diamondsuit\} \\ C &= \{1, 2, 3\} \end{aligned} \tag{1}$$

La cantidad de elementos que puede asumir o tomar un conjunto se le denomina como dominio. Una relación se ve de la manera (A, B, C), dónde estos conjuntos toman un valor específico.

Más formalemente, una relación es un subconjunto del producto cartesiano de los conjuntos denotado como:

$$A \times B \times C = \{(a, \heartsuit, 1), (a, \heartsuit, 2), (a, \heartsuit, 3), ...\} \tag{2}$$

Es decir, un subconjunto de todas las diferentes combinaciones que existen entre los conjuntos.

3.7.1. Llaves

Cada tabla, o conjunto de relaciones tiene debe de ser identificable del resto. Es debido a esto que toda tabla debe de contar con una llave primaria.

Ciertas tablas tienen una sola llave primaria posible, mientras que otras tienen varias. Cualquier posible llave primaria que no se haya utilizado como tal se le considera una llave alterna. Una llave primaria puede ser una llave simple si solo la compone una columna, o compuesta si la componen varias columnas.

No se puede que una llave primaria tome un valor nulo.

También existen las llaves foráneas, que son llaves que hacen referencia a las llaves primarias de otra relación (tabla). Estas llaves pueden ser nulas, pero si tienen un valor debe de ser un valor que haga referencia a la llave primaria de otra tupla, no se puede que apunten a algo que no existe.

3.7.2. Operaciones

Al tener llaves primarias y foráneas, las cuales tienen restricciones adicionales, es necesario contar con ciertas operaciones para no romper ninguna de las restricciones.

Cuando se quiere eliminar un registro dentro de una tabla, y ese registro tiene su llave primaria referenciada en otras tablas, es necesario seguir alguno de estas operaciones para asegurar la integridad de la información.

Se puede restristringir la operación, de modo que la operación no pueda ser realizada hasta que no exista ninguna referencia al registro.

Se puede realizar una operación en cascada, de modo que se elimine el registro y también todos los registros que hagan referencia a el mismo.

Se pueden poner todas las referencias a esa tabla como nulos, aunque es posible que con esto se rompa otra restricción.

Se puede poner un valor por defecto a los registros que referencian a esa tabla.

También se puede hace que se genere una operación predefinida por el usuario.

3.7.3. Alg Relacional

Es el álgebra que se puede efectuar en las relaciones. Es muy similas (quizá el mismo) que el álgebra en conjuntos.

La primera operación que se puede realizar es la unión. Es un operador binario que actúa en relaciones con un mismo dominio y grado. Se denota con el símbolo $\cup$.

También existe la intersección, que también es un operador binario que requiere de un mismo domino y grado. Se denota con el símbolo $\cap$.

La diferencia se refiere a un operador binario que de igual manera requiere de un mismo dominio y grado, que regresa todos los registros en la primera relación que no se encuentran en la segunda relación. Se denota con el símbolo $-$.

La proyección es un operador que filtra las columnas. Este es un operador unario en el que se especifica qué columnas se quieren obtener y solo regresa las pedidas. Se denota como π_{select}.

Existe también en operador de selección, el cual filtra los registros o tuplas de una relación. También es un operador unario. Se le denota como σ_{cond}.

Hay dos tipos de operadores especiales. El primero es el join natural. Es un operador binario cuya única restricción es que exista al menos una columna igual en ambas relaciones. Este operador devuelve todas las tuplas en las que la columna igual coincide en valor. Se le denota con el símbolo ><.

Finalmente existe el teta join, el cual es un operador binario que no tiene restricciones. Sin embargo, el teta join requiere de una condición, la cual debe de relacionar ambar tablas. Se denota con la misma notación que el join natural, pero se le indica la condición en el subíndice.

¿Cómo se especifíca el teta join en SQL?

3.8. SQL

El sintáxis básico de una consulta en SQL es el siguiente:

```
SELECT columnas
FROM tablas
WHERE condicion
```

También se puede hacer uso de funciones agregadas, las cuales nos dan una estructura como la siguiente:

```
SELECT columnas
FROM tablas
WHERE condicion
GROUP BY columnas que agrupan
HAVING condiciones sobre funciones agregadas
ORDER BY columna por la que ordenar
```

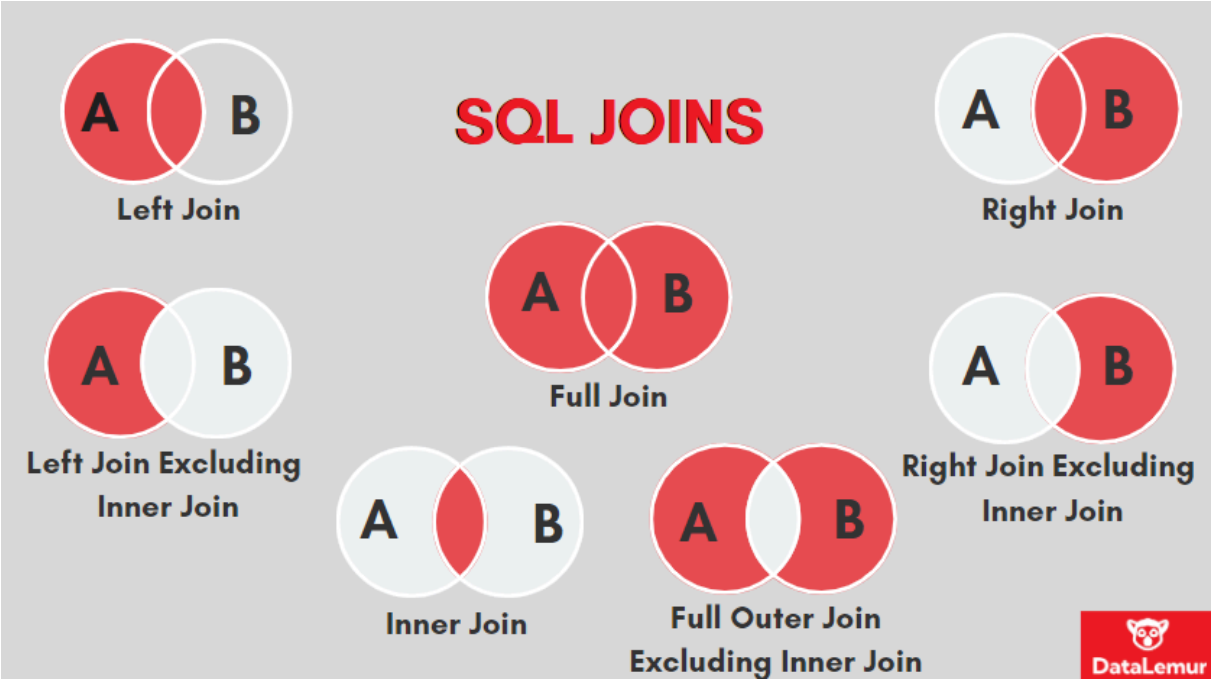
Existe una diferencia entre where y having. Where filtra los registros de la tabla, mientras que having filtra las funciones agregadas.

Las funciones agregadar que existen son las siguientes:

```
SUM(expresión) Suma de la expresión.
AVG(expresión) Promedio de la expresión
MIN(expresión) Mínimo de la expresión
MAX(expresión) Máximo de la expresión
COUNT(*) Número de tuplas (renglones)
COUNT(expresión) número de renglones en los que la expresión es no nula.
STD(expresión) Desviación estándar de la expresión
```

Existen generalmente dos maneras de replicar los joins de álgebra relacional en SQL. La primera forma es utilizando la clausula where para especificar que se busca que las llaves foráneas coincidan.

La segunda forma es utilizando la sintáxis especial para los joins. Cabe aclarar que no todos los tipos de joins están disponibles en todos los DBMSs.



¿Qué funciones extra existen en sql, como las funciones now, etc?

3.9. Stored Procedures and Functions

Dentro de un DBMS se pueden tener stored procedures y stored functions.

Como sus nombres indican, estos son funciones y procedimientos que son almacenados dentro del DBMS, y que pueden ser invocados posteriormente. Naturalmente, antes de entender la ventaja de estos, es importante entender la diferencia entre un procedimiento y una funcion.

3.9.1. Funciones vs Procedimientos

La funcion es un concepto basico en matematicas, que es un mapeo entre un dominio y una imagen. Esta definicion no es del todo util para el programador, sin embargo es necesario entender que, por definicion, exite una transformacion. Una funcion requiere de tanto un dominio como una imagen, y la transformacion actua sobre el dominio para producir la imagen.

$$f : D \mapsto I \tag{3}$$

Esto quiere decir que para cada elemento dentro de D, existe otro elemento dentro de I tal que $f(d) = i$.

Naturalmente podemos concluir que una funcion es aquella que tiene ambos una entrada y una salida. Este concepto es muy utilizado dentro del paradigma de la programacion funcional.

Por otro lado, un procedimiento es aquel no no necesariamente tiene una entrada o salida. Es posiblemente la primera definicion de funcion que se le da a una persona al aprender a programar, y es la manera principal en la que los programadores reutilizan codigo.

3.9.2. Ventajas y Desventajas

Personalmente siento que las ventajas de utilizar stored procedures son pocas, y deberian ser utilizadas como excepcion y no como regla.

La primera ventaja es que son mas eficientes, o al menos ligeramente mas eficientes. Esto debido a dos cosas.

Primero, son menos costosas para enviar a traves de la red, esto debido a que la cantidad de informacion disminuye considerablemente. Debido a que estas instrucciones se encuentran dentro del DBMS, solo se debe de transmitir la llamada o invocacion al stored procedure/function, y este solo envia de vuelta el resultado. Esto a comparacion de enviar una query completa a la base de datos.

Segundo, cuando el DBMS recibe una query, tienen que interpretar (compilar y ejecutar) la query. Sin embargo, muchos DBMS compilan las queries, por lo que cuando llega una invocacion a al procedimiento, este ya no debe de ser compilado y puede ser ejecutado directamente.

Y la segunda ventaja que veo, es que se puede restringir el acceso a ciertos usuarios. Puede ser que algun equipo tenga acceso a ciertos stored procedures que actuan en ciertas tablas, pero ellos no tienen acceso a las tablas mismas. Los stored procedures nos ayudan a tener una mejor seguridad de acceso a la base de datos.

Sin embargo, personalmente veo muchas desventajas tambien.

Delegar reglas y logica del negocio a la base de datos no me parece la mejor decision arquitectonica. Esto tambien parece que seria mas dificil de probar. Asi mismo, el equipo puede olvidar que estos procedimientos existen y pueden generar bugs que serian muy complicados de encontrar.

Realmente no provee un encapsulamiento necesario. Una aplicacion bien hecha ya deberia de tener una capa de encapsulamiento para la base de datos, ya sea un modelo o un repositorio, por lo que el encapsulamiento deberia suceder ahi, y no dentro de la base de datos.

Esto es menos mantenible. Primero, se requiere estar checando el DBMS. No solo eso, sino que la aplicacion se vuelve dependiente del DBMS, lo que causa que no se pueda cambiar a otra base de datos sin tener que generar muchas migraciones y cambiar muchos stored procedures, ya que seguramente las sintaxis y forma de operar no son las mismas.

3.10. Transacciones

Es una de las habilidades y funcionalidades fundamentales de un gestor de base de datos, y es la herramienta que nos permite mantener la integridad de nuestros datos y de una manera correcta.

En un alto nivel, las transacciones realizan prodecimientos, ya sean queries a la base de datos, inserciones, o incluso stored procedures, de una manera atomica. Esto quiere decir que, todas estas instrucciones son tomadas como una sola entidad, un atomo, y solo se efectua si todas fueron exitosas, y de no ser asi se realiza un rollback, lo que nos permite mantener nuestra base de datos en un estado correcto.

Mas formalmente, las transacciones estan definidas por las propiedades conocidas como ACID (atomicidad, coherencia, aislamiento y durabilidad).

3.10.1. Atomicidad

Se refiere a que la transaccion es una unidad atomica de trabajo, es decir, no se puede dividir, por lo que actua como una unidad. Pare efectos practicos significa que, o se realzan todas las tareas dentro de la transaccion, o no se realiza ninguna.

3.10.2. Coherencia

Cuando una transaccion termina, la base de datos debe de quedar en un estado coherente. Esto quiere decir que todas las reglas transaccionales y mecanismos internos como indices de arbol B o listas doblemente ligadas deben de terminar en un estado correcto y valido al finalizar la transaccion.

3.10.3. Aislamiento

Si existe otro proceso que requiere ver de leer o escribir a la base de datos, este solo puede ver el estado antes de la transaccion o despues, pero no durante. Esto quiere decir que la transaccion no expone el estado de la base de datos hasta que esta haya sido terminada, ya que de no ser asi se podria operar en un estado invalido.

3.10.4. Durabilidad

Una vez que la transaccion termina, los efectos se "commitean" y se hacen de una forma permanente.

realmente no tengo dudas sobre las transacciones

3.11. Normalizacion

La idea de este concepto es eliminar lo mas posible las anomalias dentro de una base de datos, lo cual permite que esta sea modificada mas facilmente. Se mantiene la integridad de los datos para asegurarse que la base de datos no puede estar en un estado invalido, o que no corresponde con el mundo real.

Existen pasos para normalizar una base de datos, dichos pasos dados por las formas normales. Sin embargo, una base de datos bien diseñada no deberia de requerir de normalizacion. Solo una base de datos que no fue construida de manera correcta es normalizada. Con experiencia es posible crear bases de datos que naturalmente ya son normales.

Sin embargo, tambien es importante conocer las caracteristicas de las primeras tres formas normales para observar y decidir si mi base de datos es correcta o falta mejorarla.

3.11.1. 1FN

Esta es la primera forma normal. Se caracteriza por tener atributos que poseen valores simples y no tienen varios valores por campo. Las columnas tienen nombres unicos y los registros no son los mismos o duplicados.

Es muy natural que las bases de datos ya se encuentren en esta primera forma normal, y de no serlo no es tan complicado llegar a esta forma.

3.11.2. 2FN

Es en la que existe una dependencia funcional, en la que todos los atributos de una relacion dependen de un atributo en especifico.

Esto parece un poco complejo, pero realmente se refiere a que todas las entidades cuentan con una llave primaria, de la cual todos los demas atributos dependen, ya que es la que los identifica con respecto al resto de registros.

3.11.3. 3FN

Esta tercera forma normal busca eliminar las dependencias transitivas. Esencialmente, en el paso pasado se definio que todas las entidades deben de tener una llave primaria que las identifique. Este paso lo que busca es asegurarse que todos los atributos de una entidad en particular dependan, o se refieran, a la llave primaria.

Es decir, debe de existir una relacion semantica directa entre una llave primaria y sus atributos.

3.11.4. Demas Formas

Existen mas formas normales, sin embargo, no creo que se crucial entederlas. Es importante conocer que existen, pero para la mayoria de los casos estas no son relevantes, y con tener una base de datos en la tercera forma normal es mas que suficiente.

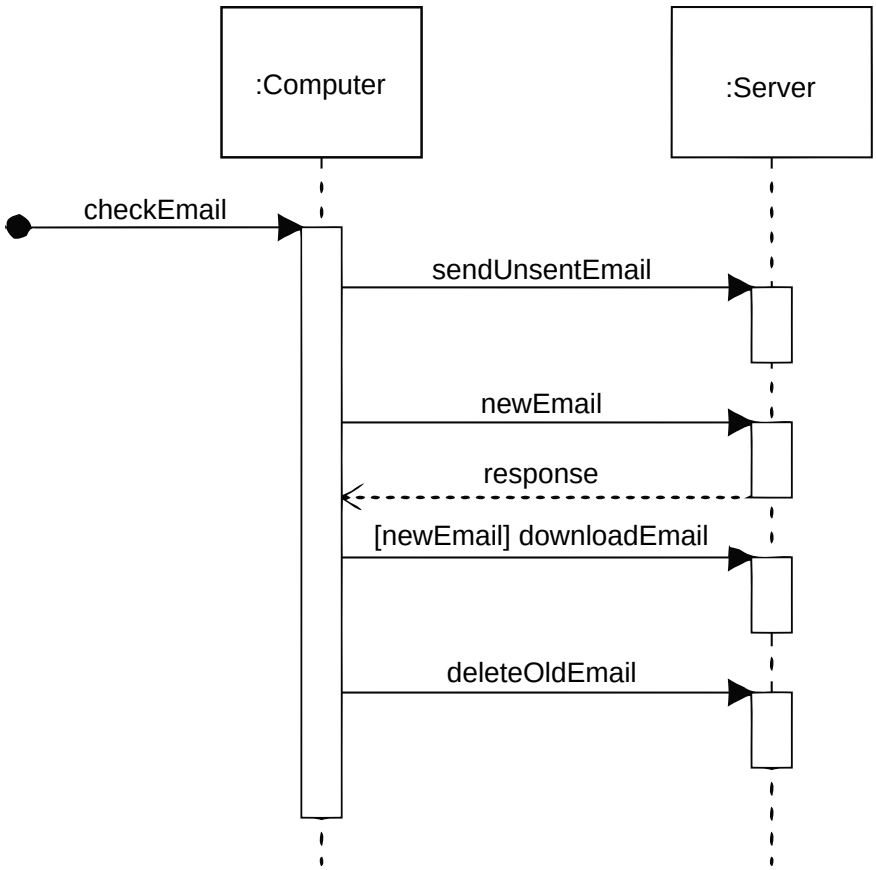
Cuando es valioso pasar de la tercera forma normal? En que tipo de proyecto?

4. UML

4.1. Diagramas de Secuencia

Es un diagrama de interacción especial que muestra una interacción o un conjunto de objetos con sus relaciones y los mensajes que hay entre ellos. Tiene un énfasis en una ordenación temporal.

Este diagrama está compuesto por dos dimensiones: en la vertical se encuentra la dimensión temporal, y en la horizontal se encuentran las instancias de los objetos.

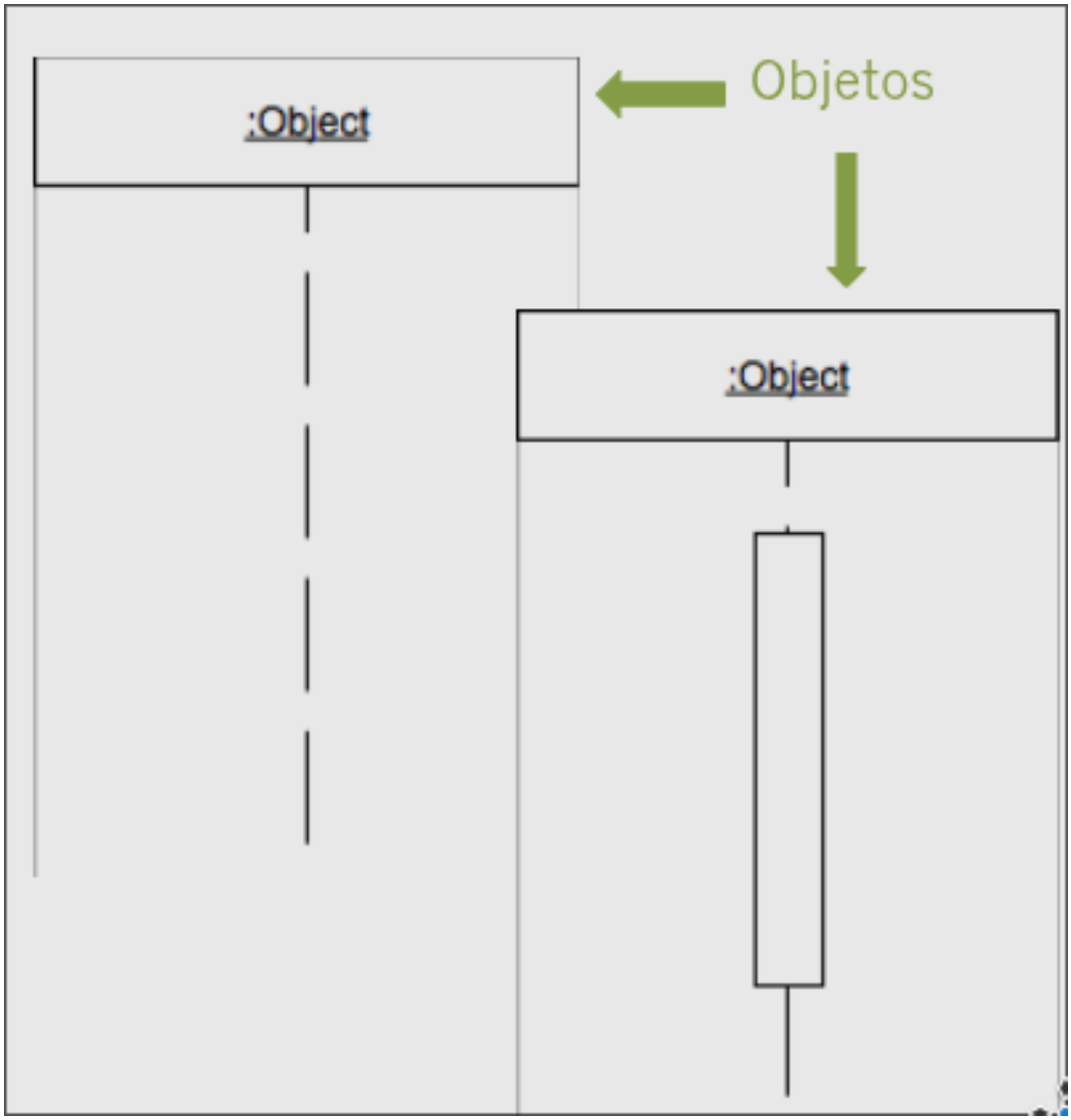


Nosotros utilizaremos el diagrama de secuencia para describir los métodos particulares de una clase. A diferencia de un diagrama de casos de uso, el diagrama de secuencia contiene el detalle de la implementación, lo que incluye los **objetos** que se utilizarán y los **mensajes** intercambiados por los objetos.

Estos pueden ser utilizados de dos formas, de instancia y genéricos. De instancia describen un caso de uso, y genéricos la interacción entre varios.

4.1.1. Objetos

Se representan en un rectángulo, con el símbolo `:` como prefijo. Cuentan con una línea discontínua vertical que denota la existencia del objeto, y se le puede superponer un rectángulo para indicar que en esa ventana de tiempo el objeto ejecuta una acción, es decir que tiene el control.

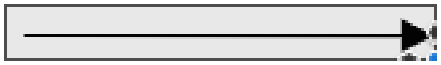


Dentro de un flujo se puede especificar la creación o destrucción de un objeto con las palabras `<<creates>>` y `<<destroys>>`.

4.1.2. Flujos

Existen tre tipos de flujos, que se representan con flechas.

1. El flujo síncrono, con una flecha continua con cabeza llena.

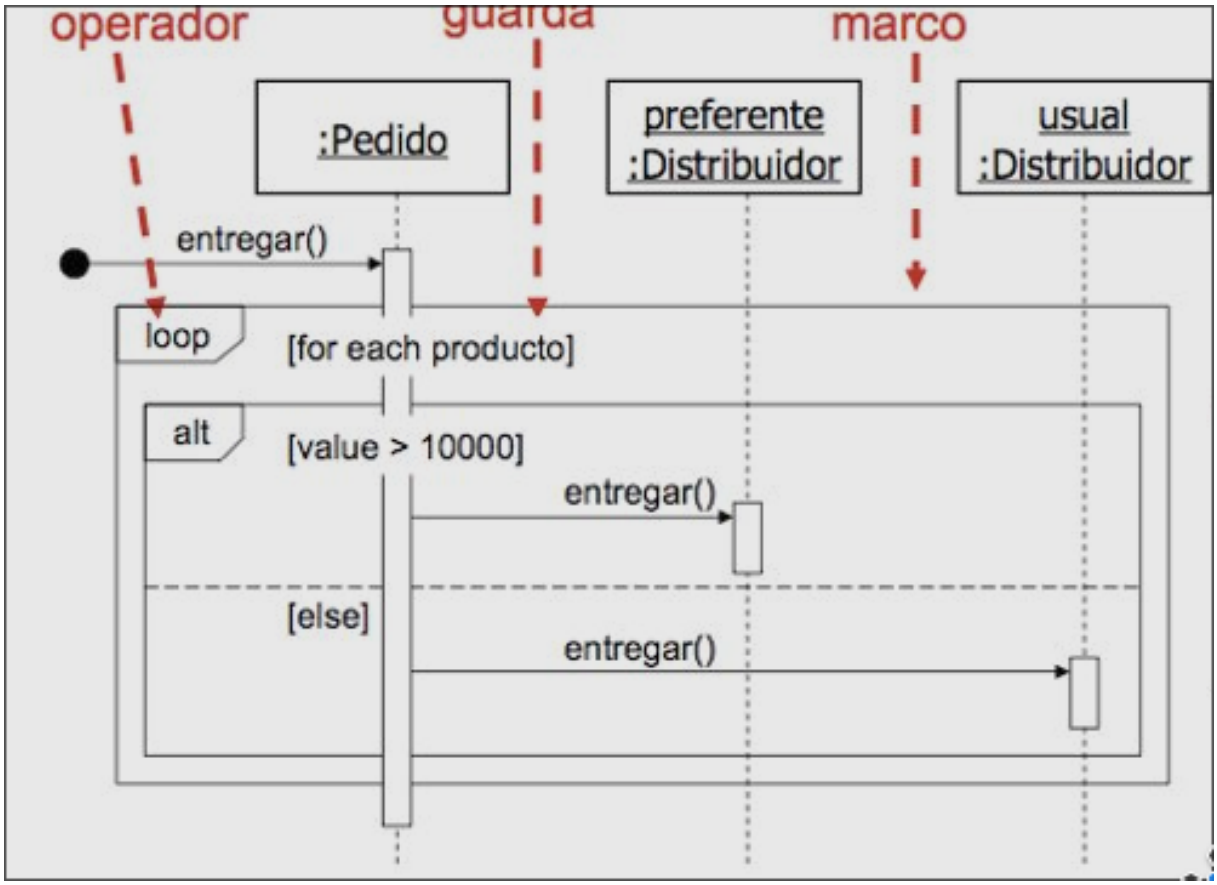


2. El flujo asíncrono, con una flecha contínua con cabeza abierta.

3. El retorno de una llamada, con una flecha discontínua con cabeza abierta.

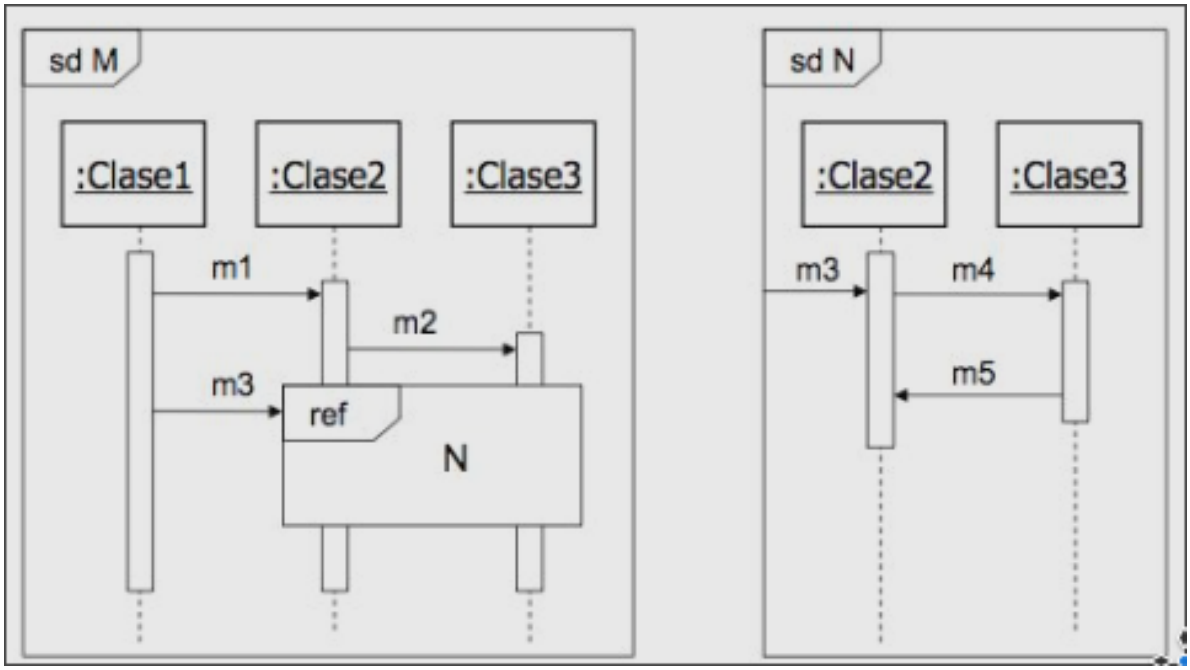
4.1.3. Operadores

Existen varios operadores y fragmentos combinados dentro de este tipo de diagramas. Se componen del operador, la guarda y el marco.



Los operadores más comunes son

- Alternativa (alt): Modela un if else. La elección se pone en la guarda.
- Opción (opt): Equivale a un alt pero solo con un fragmento. Modela un switch.
- Bucle (loop): El fragmento se ejecuta varias veces, la guarda indica la condición.
- Diagrama de Secuencia (sd): Le da un nombre al diagrama.
- Referencia (reg): Se refiere a otro diagrama de secuencia aparte.



- Paralelo (par): cada fragmento sucede en paralelo.
- Región crítica (critical): Solo se puede tener un proceso ejecutando.

4.2. Generar Pruebas a Partir de Casos de Uso

Para poder generar pruebas a partir de casos de uso es importante primero tener los casos de uso bien identificados, pero también haberlos descrito e incluso detallado.

A esto se refiere la descripción que incluye lo las pre y post condiciones, los puntos de extensión, etc. Al detalle se refiere el flujo de actividades que va a seguir en el caso de uso. Esto puede ser en texto, o bien utilizando un diagrama de actividades.

Adicionalmente a esto, se deben de identificar todos los escenarios por los que un usuario puede ir dentro del caso de uso. Con esto se refiere a identificar las rutas que se pueden seguir durate el caso de uso, que combinación de rutas entre el flujo básico y los flujos alternos existen, de modo que se tengan contempladas todas las rutas y combinaciones entre flujos posibles.

Ahora sí, teniendo estas cosas se pueden generar las pruebas. El proceso se divide en tres pasos:

- Generar todos los escenarios
- Por cada escenario identificar al menos una prueba y las condiciones que hacen que se ejecute
- Por cada prueba identificar los valores para la prueba

4.2.1. General los Escenarios

Este paso es redundante si ya se identificaron previamente, pero en caso de que no este es el momento para identificar los escenarios dentro de un caso de uso.

4.2.2. Identificar los Casos de Prueba

En este paso se identifican los casos de prueba. Esencialmente, se toma cada escenario y se juega con las posibilidades. ¿Qué pasa si esta condición no se ha complido? ¿Si el usuario no está autenticado? etc. Es útil utilizar una matríz con las condiciones. En este paso solo se identifica si las condiciones son válidas, inválidas, o si no es aplicable. Todavía no se le dan valores específicos.

Test Case ID	Scenario/ Condition	Student ID	Password	Courses selected	Prerequisites fulfilled	Course Open	Schedule Open	Expected Result
RC 1	Scenario 1- successful registration	V	V	V	V	V	V	Schedule and confirmation number displayed
RC 2	Scenario 2- unidentified student	I	N/A	N/A	N/A	N/A	N/A	Error message; back to login screen
RC 3	Scenario 3- valid user quits	V	V	N/A	N/A	N/A	N/A	Login screen appears
RC 4	Scenario 4- course registration system unavailable	V	V	N/A	N/A	N/A	N/A	Error message; back to step 2
RC 5	Scenario 5- registration closed	V	V	N/A	N/A	N/A	N/A	Error message; back to step 2

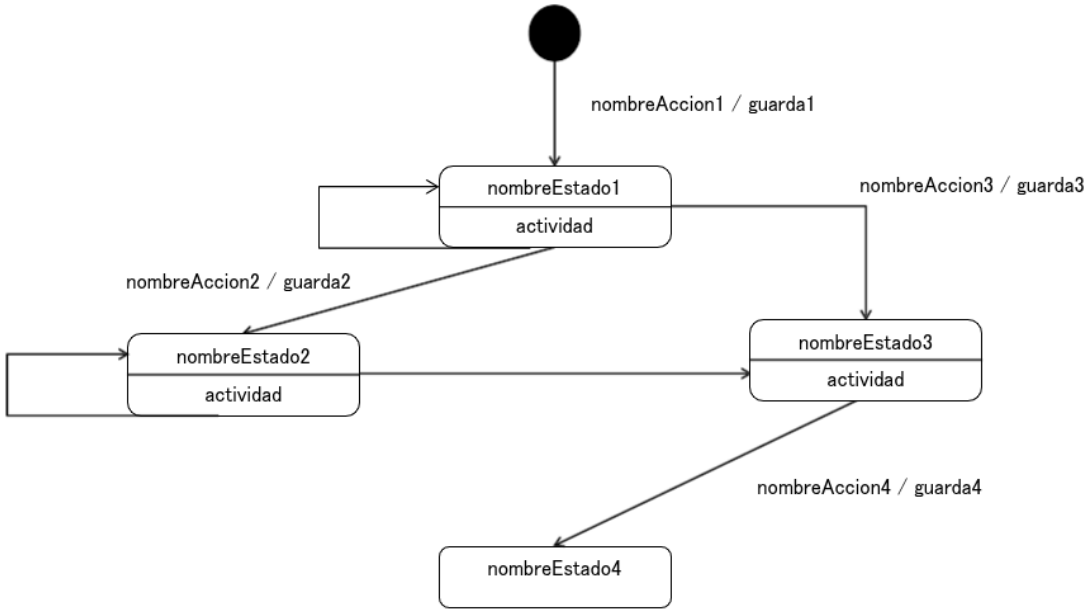
¿Qué tan relevante es esto hoy en día? La información es de hace más de 20 años.

4.2.3. Identificar Valores

Finalmente, el último paso es darle valores específicos a todas las Vs e Is, para tener valores concretos con los que se puedan realizar las pruebas.

4.3. Diagrama de Estado

Es un poco redundante escribir mas sobre diagramas de estado. Sin embargo, es importante al menos mostrar la sintaxis para estos.



Por definicion, un automata debe de tener un estado inicial. Este es denotado por el circulo negro.

A las transiciones entre etados se les denomina eventos, y de igual manera se debe de especificar cerca de la flecha.

Finalmente, se le puede asignar una actividad a cada estado. Una actividad es una accion que se esta realizando dentro de tal estado.

Cabe aclarar que dentro de la clase no vimos si un automata debe de tener estados finales o aceptantes. Sin embargo, me parece deberian de tener, ya que, tomado como ejemplo una transaccion dentro de una base de datos, puede que esta pase por muchos estado, pero finalmente queremos que termine en un estado valido, de otra manera la transaccion no podria ser completada.

Existe una forma de denotar estados de aceptacion?

4.4. Diagramas de Despliegue

Estos diagramas representan la arquitectura de ejecucion de un sistema.

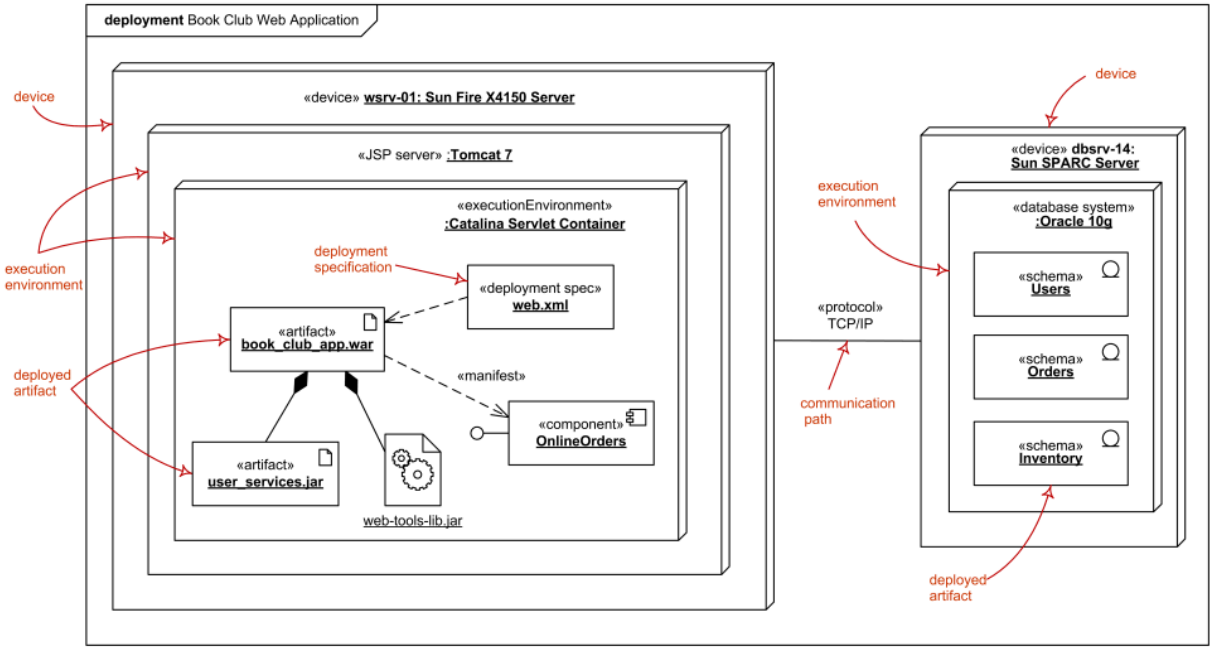
Tienen dos componentes, los nodos y los artefactos.

Los nodos son los tipos de recursos computacionales que se pueden tener, y sobre los que se pueden desplegar artefactos, que veremos mas adelante. Para fines de la materia nos enfocamos en dos posibles estereotipos de nodos: los dispositivos, denotados como `<<device>>`, y los entornos de ejecucion, denotados como `<<execution environment>>`.

Estos nodos pueden albergar otros nodos dentro, de modo que un dispositivo pueda albergar un entorno de ejecucion.

Los artefactos representan la especificacion de un elemento concreto y real. Usualmente son los archivos, como los ejecutables, documentos, archivos de configuracion, etc. Tambien pueden ser las tablas de la base de datos, etc. Los artefactos son los que se despliegan dentro de los nodos.

Adicionalmente, tambien tenemos lineas que indican vias de comunicacion entre nodos. En estas lineas se debe de especificar el metodo o protocolo por el cual se estan comunicando.



5. Entrevistas

Necesitan contener el marco teórico, la aplicación del concepto en mi proyecto, y las áreas de oportunidad que hubieron.

Index of Figures

Figure 1 13
Figure 2 23
Figure 3 27
Figure 4 28
Figure 5 28
Figure 6 28
Figure 7 28
Figure 8 29
Figure 9 30
Figure 10 31
Figure 11 32
Figure 12 33